

02-510/710: Computational Genomics, Spring 2026

HW4: Read Mapping/Alignment, BWT, HMM

Version: 1

Due: 1:30 EST, Apr 06, 2026 on Canvas

Topics in this assignment:

1. Local Sequence Alignment
2. Burrows-Wheeler Transform
3. Hiddn Markov Model

What to hand in.

- One write-up (in PDF format) addressing each of the questions.
- All source code. If the skeleton is provided, you just need to complete the script and send it back. Your code is tested by autograder, please be careful with your main script name and output format.

Submit the following files as a single `.zip` file to Canvas:

`./S2026HW4.pdf`
`./SW.py`
`./BWT.py`

Please note that all the solutions and code must be your own. We will check for plagiarism after the final submission.

1. [25 points] Local Sequence Alignment

You should implement the dynamic programming from scratch in Python 3.

Complete the function `smith_waterman` in the python script `SW.py` to implement the Smith-Waterman algorithm with the scoring function below:

Match:	2
Mismatch:	-2
Gap:	-1

Note: Your code is graded by an autograder. You may use the `numpy` Python package if desired. The script should be able to run with command line:

```
python SW.py example.fasta
```

The `smith_waterman` function should return 3 values: the alignment score, the alignment in the first sequence, and the alignment in the second sequence. The alignment score should be an integer, and the two alignments should be strings with no whitespace. Use a dash '-' to indicate gaps.

See the following snippet of code for an example:

```
> > score, align1, align2 = smith_waterman('GGTTGACTA',
'TGTTACGG')
> > print(score, '\n', align1, '\n', align2)
9
GTTGAC
GTT-AC
```

Hint: You may have to specifically test for edge cases (e.g., allowing for the output to begin with an indel (gap), handling multiple indels in a row, handling inputs that differ drastically in length).

2. [25 points] Burrows-Wheeler Transform

In this problem, you will complete several functions in `BWT.py`, related to the Burrows-Wheeler Transform (BWT), which will be submitted to the autograder, in addition to your short responses below.

- (a) The Domestic cat genome has about 2,500 million base pairs. How many gigabytes would it take to store this information (Hint: first determine how many bits you need to encode a nucleotide; 1 gigabyte = 1073741824 bytes)? A Fruit fly has a genome size of 122,653,977 base pairs; how many gigabytes would you need to store it?

Solution

Genomes are quite large! Compressing the genome to be as small as possible, without losing any information, is vital for efficient storage and computation. Here we will explore a simple, reversible compression technique called Run Length Encoding (RLE). If a character is repeated more than once, we replace it with two instances of that character followed by the length of the run. If it only occurs once, we leave it alone. For example:

$$RLE(\text{"WAAAAHAAAHAAAA!"}) = \text{"WAA4HAA3HAA4!"}$$

- (b) Complete the `rle` function in `BWT.py` to return the run-length-encoded version of an input string. **This must be your own code, you may not take code from elsewhere.** Use your function to encode the string:
`"mississippiicecreammississippi"`
How long is the encoded string?

Solution

BWT is a technique to transform a sequence into one that has more repeated runs of characters. The transform is lossless, and easily reversible. As we discussed in class, this helps when trying to search for a substring. It can also help when performing lossless compression. See the Wikipedia [article](#) for more details. **Feel free to use their python implementation for your code below.**

- (c) Complete the `bwt_encode` function in `BWT.py` to return the transformed version of an input string (You must as a first step insert “{” and “}” at the beginning and end of the input string, respectively. You may assume that these two characters will not appear in any raw input strings).
(d) Complete the `bwt_decode` function in `BWT.py` to return the original string given the BWT version as input (The input BWT string will contain the “{” and “}” delimiters, but your decoded output should not).

For questions 2c and 2d below, assume you are using the most straightforward, basic implementation of BWT, and that anytime you need to sort, you use Merge Sort. Note that the complexity of sorting K integers might be different than the complexity of sorting K strings of length N . Typically, the complexity of a sorting algorithm is written as a function of the number of items to be sorted, and is given in terms of the number of comparisons, but assumes that the length of each item is a constant (i.e. $K \gg N$). This assumption means that each comparison is a constant-time operation. Is this true for situations when BWT needs to sort? You should see that here the aforementioned assumption does not hold (i.e. $K=N$).

- (e) For a string of length N , what is the worst case runtime complexity of `bwt_encode`?

Solution

- (f) For a string of length N , what is the worst case runtime complexity of `bwt_decode`?

Solution

- (g) Now compute `RLE(BWT(<same string from 2b>))`. How long is the encoded string now?

Solution

- (h) Explain in your own words how BWT makes it so that the transformed strings have long runs of identical characters.

Solution

- (i) Finally, explore using the BWT followed by RLE for various strings (each of your examples should be at most 100 characters long, and you can use the full alphabet, save for “{” and “}”, and **must not be trivial copies of the two given strings in the file**):

- i. Provide an example in which the final string is longer than the original string and include the lengths of both for comparison.

Solution

- ii. Provide an example in which the final string is shorter than the original string and include the lengths of both for comparison.

Solution

- iii. What type of strings seem to compress better with `rle(bwt(s))`? What are these called in DNA?

Solution

In practice, even more compression is applied after `RLE(BWT(s))`, and there are clever ways to search for matches against a string while it is compressed. BowTie (22784 citations as of 01/26/2024) and BowTie2 (44091 citations as of 01/26/2024) are famous DNA alignment tools that make use of these techniques, making it possible to align millions of DNA reads to the genome within hours on a laptop.

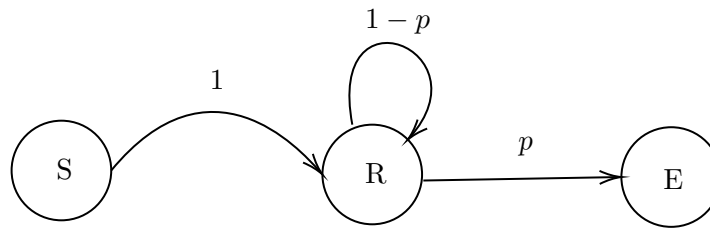


Figure 1: Random Genome HMM

3. [30 points] Hidden Markov Models

Warm-Up

- (a) Consider the state transition diagram for the very simple HMM shown in Figure 1. The state S is a silent start state, and the state E is a silent end state. The state R , which stands for *random*, emits nucleotides with the following probabilities:

nucleotide	emission probability
A	0.20
C	0.30
G	0.30
T	0.20

The human genome is approximately 3.2 billion base pairs long. Suppose we generate a genome using the HMM in Figure 1. Find the value of p such that the expected length of this random genome is equal to the size of the human genome.

Solution

- (b) Find the expected GC content of the genome generated in the previous part (write the answer as a percentage).

Solution

Supervised Learning with HMMs

- (c) Consider supervised learning for a Hidden Markov Model (HMM). We are given n labeled training examples. For the k -th example, let

$$O^{(k)} = (o_1^{(k)}, \dots, o_{T_k}^{(k)})$$

be the observation sequence (e.g., histone mark signals), and

$$Q^{(k)} = (q_1^{(k)}, \dots, q_{T_k}^{(k)})$$

be the corresponding sequence of hidden states (chromatin states), where each $q_i^{(k)} \in Q$.

The HMM parameters are:

$$A = \{a_{ij}\}, \quad a_{ij} = P(q_t = q_j \mid q_{t-1} = q_i),$$

$$B = \{b_j(o)\}, \quad b_j(o) = P(o_t = o \mid q_t = q_j).$$

Show that the maximum likelihood estimates of the transition and emission probabilities are given by:

$$\hat{a}_{ij} = \frac{\sum_{k=1}^n \sum_{t=2}^{T_k} \mathbb{I}\{q_{t-1}^{(k)} = q_i, q_t^{(k)} = q_j\}}{\sum_{k=1}^n \sum_{t=2}^{T_k} \mathbb{I}\{q_{t-1}^{(k)} = q_i\}},$$

and

$$\hat{b}_j(o) = \frac{\sum_{k=1}^n \sum_{t=1}^{T_k} \mathbb{I}\{q_t^{(k)} = q_j, o_t^{(k)} = o\}}{\sum_{k=1}^n \sum_{t=1}^{T_k} \mathbb{I}\{q_t^{(k)} = q_j\}}.$$

Here, $\mathbb{I}\{\cdot\}$ is the indicator function.

Solution

HMMs Application

HMMs Application: Chromatin State Annotation

- (d) Chromatin states are functional genomic regions such as promoters, enhancers, and repressed regions. These states can be modeled using an HMM, where hidden states correspond to chromatin states and observations correspond to histone modification signals (e.g., H3K4me3, H3K27ac). You are given labeled training data consisting of genomic regions where: 1) the chromatin state at each position is known, and 2) the observed histone mark signals are provided.

Design an HMM for this task by specifying: - the hidden states, - the observation space, - the structure of the transition matrix A , - the emission model B .

What method would you use to estimate the transition and emission probabilities?

Solution

- (e) Now suppose you are given a new genomic sequence with histone mark observations but without chromatin state labels. You want to infer the most likely chromatin state at each position. List two methods you could use for this task and briefly explain how they differ.

Solution

- (f) In practice, chromatin states can vary across cell types (e.g., enhancer patterns differ between cell lines). Suppose you have two sets of unlabeled data from two different cell types.

You want to learn separate HMMs (with the same structure but different parameters) for each cell type.

What method would you use to estimate the transition and emission probabilities in this setting, and why is it appropriate?

Solution

- (g) After training two HMMs for two different cell types, you are given a new genomic region with histone mark observations. You want to determine which cell type the sequence most likely comes from.

What method would you use to perform this classification?

Solution